

// Jivops

Ollama y LLMs Self-Hosted

Guía completa — 30 lecciones

Fundamentos · Gestión de modelos · API y desarrollo · Despliegue en
contenedores · Rendimiento · Producción avanzada

Contenido

Módulo 1: Fundamentos

Lección 01 — Qué es Ollama y por qué self-hosted LLMs

Lección 02 — Instalación de Ollama

Lección 03 — Modelos GGUF y cuantización

Lección 04 — Requisitos de hardware — CPU vs GPU

Lección 05 — Ollama vs servicios cloud de LLM

Módulo 2: Gestión de modelos

Lección 06 — Descargar y ejecutar modelos con Ollama

Lección 07 — Biblioteca de modelos de Ollama

Lección 08 — Modelfile — personalizar un modelo

Lección 09 — Ajustar parámetros de inferencia (temperature, context)

Lección 10 — Gestión de espacio en disco y múltiples modelos

Módulo 3: API y desarrollo

Lección 11 — API REST de Ollama

Lección 12 — Streaming de respuestas con la API

Lección 13 — Compatibilidad con API de OpenAI

Lección 14 — Embeddings con Ollama

Lección 15 — Integración con LangChain y frameworks

Módulo 4: Despliegue en contenedores

Lección 16 — Ollama en Docker

Lección 17 — Persistencia de modelos en contenedores

Lección 18 — Ollama en Kubernetes

Lección 19 — Exponer Ollama de forma segura (reverse proxy, auth)

Lección 20 — Escalado de Ollama — múltiples réplicas

Módulo 5: Rendimiento y optimización

Lección 21 — Optimización de rendimiento de inferencia local

Lección 22 — Offloading de capas entre CPU y GPU

Lección 23 — Gestión de memoria y contexto largo

Lección 24 — Batch de peticiones concurrentes

Lección 25 — Comparación de modelos — tamaño vs calidad vs velocidad

Módulo 6: Producción avanzada

Lección 26 — Seguridad de un despliegue self-hosted de LLM

Lección 27 — Monitoreo de Ollama en producción

Lección 28 — Actualización de modelos sin downtime

Lección 29 — Casos de uso — cuándo self-hosted vale la pena

Lección 30 — Proyecto final: plataforma de LLMs self-hosted lista para producción

Módulo 1: Fundamentos

Lección 01 — Qué es Ollama y por qué self-hosted LLMs

Empaqueta la descarga, gestión y ejecución de modelos de lenguaje en una herramienta simple con comandos directos, abstrayendo la complejidad de configurar manualmente el runtime de inferencia y sus dependencias.

Al ejecutar el modelo completamente dentro de su propia infraestructura, la empresa garantiza que los datos sensibles nunca salen hacia un servidor externo de terceros, algo importante para cumplir requisitos de privacidad o regulación específicos.

Lección 02 — Instalación de Ollama

Un servicio en segundo plano (daemon) que gestiona la descarga, carga en memoria y ejecución de los modelos, exponiendo una API local para interactuar con ellos.

Al correr como un servicio continuo en segundo plano, cualquier aplicación (un script, una herramienta de terceros) puede enviar peticiones a Ollama en cualquier momento sin depender de que alguien tenga una ventana o programa específico abierto manualmente en ese instante.

Lección 03 — Modelos GGUF y cuantización

Un formato de archivo optimizado para almacenar modelos cuantizados de forma eficiente, diseñado específicamente para facilitar la inferencia rápida en hardware de consumo con recursos limitados.

Al representar cada peso del modelo con menos bits (por ejemplo, 4 bits en vez de 16 o 32), el tamaño total del modelo en memoria se reduce significativamente, permitiendo que quepa en el hardware limitado de una laptop, mientras la versión de precisión completa ocuparía mucho más espacio del disponible.

Lección 04 — Requisitos de hardware — CPU vs GPU

Una GPU puede procesar las operaciones matriciales masivas de la inferencia mucho más rápido que una CPU, resultando en una generación de texto significativamente más veloz.

Las GPUs están diseñadas específicamente para realizar en paralelo las operaciones matriciales masivas que requiere la inferencia de un LLM, mientras una CPU procesa esas mismas operaciones de forma mucho más secuencial y limitada, resultando en una diferencia de velocidad significativa para exactamente la misma carga de trabajo.

Lección 05 — Ollama vs servicios cloud de LLM

Cuando se necesita control total sobre los datos, no depender de conectividad a internet, o evitar el costo variable por token en un escenario de uso muy intensivo y predecible.

Con un volumen de uso muy alto y constante, el costo variable acumulado de pagar por cada token procesado en una API externa podría superar significativamente el costo fijo de mantener infraestructura propia con Ollama, especialmente a partir de cierto punto de equilibrio de volumen de uso.

Módulo 2: Gestión de modelos

Lección 06 — Descargar y ejecutar modelos con Ollama

Descarga automáticamente el modelo desde la biblioteca de Ollama si no existe localmente, y luego inicia una sesión interactiva de chat con ese modelo ya cargado.

Un usuario nuevo puede simplemente ejecutar un único comando sin necesitar conocer de antemano que hay un paso de descarga separado del paso de ejecución, reduciendo la fricción inicial de aprender el flujo completo de la herramienta antes de poder probarla.

Lección 07 — Biblioteca de modelos de Ollama

Múltiples familias de modelos open-source distintos (como Llama, Mistral, entre otros), en diferentes tamaños y niveles de cuantización, listos para descargar con un solo comando.

Un usuario con hardware limitado puede optar por una versión más pequeña del modelo que sí quepa en su memoria disponible, aceptando cierta pérdida de calidad, mientras alguien con hardware más potente puede ejecutar la versión más grande y de mayor calidad del mismo modelo, adaptando la elección a sus recursos reales disponibles.

Lección 08 — Modelfile — personalizar un modelo

Definir un prompt de sistema personalizado, ajustar parámetros por defecto (como temperature), o combinar configuraciones específicas sobre un modelo base, empaquetando todo como una nueva variante nombrada.

Al quedar guardado dentro de la variante personalizada del modelo, ese prompt de sistema se aplica automáticamente cada vez que se usa esa variante específica, evitando tener que escribirlo manualmente de nuevo cada vez que se inicia una nueva conversación con el modelo.

Lección 09 — Ajustar parámetros de inferencia (temperature, context)

Aumenta la aleatoriedad y creatividad de las respuestas generadas, haciendo que el modelo explore opciones de palabras menos probables con más frecuencia, en vez de elegir siempre la opción más predecible.

Para extraer datos estructurados de forma consistente, se necesita que el modelo elija siempre la respuesta más probable y predecible (temperature baja); para escribir contenido creativo, cierta variabilidad y exploración de opciones menos obvias (temperature más alta) puede enriquecer el resultado en vez de limitarlo.

Lección 10 — Gestión de espacio en disco y múltiples modelos

Cada modelo (especialmente las versiones más grandes) puede ocupar varios gigabytes de espacio, y tener muchos descargados simultáneamente puede agotar rápidamente el almacenamiento disponible del sistema.

Cada modelo grande puede ocupar varios gigabytes de espacio; descargar múltiples versiones distintas sin revisar cuánto espacio queda disponible podría acumularse rápidamente hasta agotar el almacenamiento del sistema, afectando incluso otras tareas cotidianas del usuario que también necesitan espacio en disco.

Módulo 3: API y desarrollo

Lección 11 — API REST de Ollama

Que aplicaciones y scripts externos envíen peticiones HTTP programáticas al modelo cargado, integrando la capacidad de generación de texto dentro de cualquier flujo de trabajo de software personalizado.

La aplicación web propia puede hacer las peticiones HTTP necesarias a la API de Ollama de forma transparente en el backend, mientras el usuario final solo interactúa con la interfaz normal de esa aplicación, sin necesitar saber nada sobre cómo funciona Ollama ni tocar la línea de comandos directamente.

Lección 12 — Streaming de respuestas con la API

El cliente recibe fragmentos de la respuesta a medida que el modelo los va generando, permitiendo mostrar texto progresivamente en vez de una espera silenciosa hasta que toda la respuesta esté lista.

Ver contenido apareciendo de inmediato confirma que el sistema está procesando activamente la petición, dando una sensación continua de progreso, mientras una pantalla en blanco sin ninguna señal generaría incertidumbre sobre si la aplicación realmente está funcionando, aunque el tiempo real de espera total sea exactamente el mismo.

Lección 13 — Compatibilidad con API de OpenAI

Permite que aplicaciones ya construidas para usar la API de OpenAI puedan apuntar hacia Ollama simplemente cambiando la URL base, sin necesitar reescribir la integración desde cero.

Como el formato de las peticiones y respuestas es compatible, el código de la aplicación que ya sabía cómo comunicarse con la API de OpenAI sigue funcionando casi sin cambios, requiriendo solo actualizar la URL de destino en vez de reescribir toda la lógica de integración desde cero.

Lección 14 — Embeddings con Ollama

Una representación numérica (vector) del significado semántico del texto, útil para tareas como búsqueda semántica o sistemas RAG, generada de forma completamente local sin depender de un servicio externo.

Si se usara un servicio externo para generar los embeddings (aunque el modelo de generación de texto final fuera local), el contenido sensible del documento igualmente pasaría por ese servicio externo durante ese proceso; generar los embeddings también localmente mantiene la privacidad de los datos en todas las etapas del sistema RAG completo, no solo en la generación final de la respuesta.

Lección 15 — Integración con LangChain y frameworks

Provee abstracciones de más alto nivel (cadenas de procesamiento, integración con bases de datos vectoriales, agentes) que se conectan con Ollama como el motor de inferencia subyacente, sin tener que construir esas piezas desde cero.

En vez de escribir manualmente toda la lógica de conectar Ollama con una base de datos vectorial, gestionar el flujo de recuperación y construcción del prompt paso a paso, LangChain ya provee esas piezas como abstracciones reutilizables, acelerando significativamente el desarrollo de la aplicación completa.

Módulo 4: Despliegue en contenedores

Lección 16 — Ollama en Docker

Aísla el entorno de Ollama y sus dependencias del resto del sistema, facilitando la portabilidad y evitando conflictos con otras instalaciones o versiones de librerías en el host.

El contenedor empaqueta todo el entorno necesario de Ollama (dependencias, configuración) de forma autocontenida, garantizando el mismo comportamiento consistente sin importar las diferencias específicas de configuración que pudiera tener cada servidor distinto donde se despliegue.

Lección 17 — Persistencia de modelos en contenedores

Sin un volumen persistente, todos los modelos descargados se perderían si el contenedor se elimina o se recrea, obligando a volver a descargarlos completos cada vez.

Si los modelos descargados (potencialmente varios gigabytes cada uno) se perdieran cada vez que el contenedor se recrea, habría que repetir esa descarga completa nuevamente, desperdiciando tiempo real y ancho de banda de forma innecesaria cada vez, algo que un volumen persistente evita completamente al mantener esos modelos disponibles entre reinicios.

Lección 18 — Ollama en Kubernetes

Configurar correctamente la solicitud de recursos GPU en el manifiesto del Pod (como nvidia.com/gpu), además de un volumen persistente para los modelos descargados.

Al vivir dentro del mismo cluster de Kubernetes, otras aplicaciones que también corren ahí pueden comunicarse con Ollama de forma directa e interna, aprovechando además toda la infraestructura de gestión, escalado y monitoreo que el cluster ya provee, en vez de operar Ollama como un sistema completamente aislado y separado.

Lección 19 — Exponer Ollama de forma segura (reverse proxy, auth)

Ollama no incluye autenticación nativa en su API por defecto, así que exponerla directamente sin protección permitiría que cualquiera con la URL pudiera usar el modelo sin ninguna restricción de acceso.

Sin ninguna capa de autenticación protegiendo el acceso, cualquier persona que descubra la URL pública del servicio podría enviar peticiones y consumir la capacidad de cómputo del servidor libremente, potencialmente generando costos de infraestructura o degradando el rendimiento disponible para el uso legítimo previsto por el propietario.

Lección 20 — Escalado de Ollama — múltiples réplicas

Desplegar múltiples réplicas de Ollama (potencialmente en distintos nodos con su propia GPU) detrás de un balanceador de carga que distribuya las peticiones entrantes entre ellas.

Una única instancia de Ollama tiene un límite máximo real de peticiones que puede procesar simultáneamente según su hardware disponible; distribuir la carga entre múltiples réplicas en distintos nodos permite que la capacidad total agregada del sistema atienda un volumen de tráfico concurrente mucho mayor del que una sola instancia sobrecargada podría manejar por sí sola.

Módulo 5: Rendimiento y optimización

Lección 21 — Optimización de rendimiento de inferencia local

El tamaño del modelo (número de parámetros) y el nivel de cuantización usado, ya que modelos más grandes o con mayor precisión requieren más cómputo por cada token generado.

Un modelo con más parámetros requiere realizar más operaciones matemáticas para generar cada token individual de la respuesta; en el mismo hardware exacto, procesar esa carga de cómputo mayor del modelo de 70B naturalmente toma más tiempo por token que la carga menor que requiere el modelo más pequeño de 7B.

Lección 22 — Offloading de capas entre CPU y GPU

Distribuye algunas capas del modelo para ejecutarse en la GPU (más rápida) y otras en la CPU (con más memoria disponible pero más lenta), permitiendo ejecutar modelos que de otra forma no cabrían completamente en la GPU.

Las capas que se ejecutan en la CPU (más lenta que la GPU para este tipo de cómputo) introducen un cuello de botella real en el proceso general; aunque el offloading hace posible ejecutar un modelo que de otra forma no cabría, el resultado es más lento que si todo el modelo pudiera procesarse completamente en la GPU más rápida, representando un tradeoff real entre viabilidad y velocidad.

Lección 23 — Gestión de memoria y contexto largo

El modelo necesita mantener en memoria la representación interna de todo el contexto de la conversación (el historial completo) para poder generar la siguiente respuesta considerando esa información previa.

A medida que la conversación se extiende y acumula más historial dentro de la ventana de contexto configurada, la cantidad de memoria necesaria para mantener toda esa información disponible para el modelo crece proporcionalmente, pudiendo eventualmente acercarse o superar el límite real de memoria disponible del sistema si la conversación es suficientemente larga.

Lección 24 — Batch de peticiones concurrentes

La instancia debe gestionar y compartir de forma eficiente los recursos limitados de cómputo (especialmente GPU) entre las distintas peticiones simultáneas, lo que puede afectar la latencia individual de cada una si el volumen es alto.

Una única instancia de Ollama tiene un límite real de capacidad de cómputo (GPU) que debe compartir entre todas las peticiones concurrentes que recibe, degradando potencialmente la latencia de cada una si el volumen es alto; distribuir esa carga entre múltiples réplicas con su propio hardware dedicado permite atender un volumen mucho mayor sin ese mismo nivel de degradación por recurso compartido.

Lección 25 — Comparación de modelos — tamaño vs calidad vs velocidad

Un modelo más grande suele ofrecer mejor calidad de respuestas pero requiere más recursos y genera texto más lentamente, mientras uno más pequeño es más rápido y liviano pero con calidad potencialmente menor.

Si la tarea es relativamente simple y no requiere el razonamiento complejo que justificaría un modelo más grande, usar ese modelo más pesado de todas formas desperdiciaría velocidad de respuesta y recursos de hardware sin necesariamente obtener un beneficio real de calidad proporcional para ese caso de uso.

específico más sencillo.

Módulo 6: Producción avanzada

Lección 26 — Seguridad de un despliegue self-hosted de LLM

Ataques de prompt injection donde un usuario malicioso intenta manipular el comportamiento del modelo mediante instrucciones incrustadas en su entrada, además del riesgo general de acceso no autorizado si la API no está protegida.

En un entorno local controlado, solo usuarios de confianza tienen acceso directo al sistema; al exponer el servicio hacia usuarios externos no confiables, se amplía significativamente la superficie de riesgo, incluyendo la posibilidad de abuso de recursos o intentos deliberados de manipular el comportamiento del modelo mediante entradas maliciosas diseñadas específicamente para ese propósito.

Lección 27 — Monitoreo de Ollama en producción

Latencia de respuesta, tokens generados por segundo, uso de memoria/GPU, y tasa de errores o timeouts en las peticiones recibidas.

Un servicio podría seguir 'corriendo' técnicamente (el proceso está activo) mientras al mismo tiempo falla silenciosamente en atender correctamente una parte significativa de las peticiones recibidas, algo que solo se detectaría monitoreando específicamente la tasa de errores y timeouts, no solo confirmando que el proceso general sigue activo.

Lección 28 — Actualización de modelos sin downtime

Descargar la nueva versión del modelo en paralelo mientras la versión anterior sigue atendiendo peticiones, y solo redirigir el tráfico hacia el nuevo modelo una vez que esté completamente listo y validado.

Mientras la nueva versión del modelo se descarga y prepara en paralelo, la versión anterior sigue completamente disponible atendiendo el tráfico real sin ninguna interrupción; solo una vez que la nueva versión está lista y validada se redirige el tráfico hacia ella, evitando el período de indisponibilidad que ocurriría si se eliminara primero la versión activa antes de tener la nueva lista.

Lección 29 — Casos de uso — cuándo self-hosted vale la pena

Datos sensibles que no pueden salir de la infraestructura propia, volumen de uso alto y predecible que justifique el costo fijo de infraestructura, y una tarea que no requiera necesariamente la máxima calidad de los modelos propietarios más grandes.

Sin un volumen de uso lo suficientemente alto como para justificar el costo fijo de mantener infraestructura propia, y sin un requisito estricto de privacidad que obligue a mantener los datos internamente, las principales ventajas de self-hosted no aplicarían realmente a ese caso, haciendo que una API externa más simple y sin esa inversión inicial sea la opción más práctica y económica para ese escenario específico.

Lección 30 — Proyecto final: plataforma de LLMs self-hosted lista para producción

Modelos apropiados descargados y personalizados con Modelfile + despliegue en contenedores con persistencia + reverse proxy con autenticación + escalado con réplicas + monitoreo de latencia y errores + estrategia de actualización sin downtime.

Intenta mencionar al menos dos pilares: modelos personalizados con Modelfile, despliegue en contenedores con persistencia, reverse proxy con autenticación o monitoreo con estrategia de

actualización sin downtime.